

Booking & Admin Dashboard for a Spa + Clinic

A complete specification for turning a booking-request form into a real scheduling system — data model, admin screens, the availability engine, reporting, and front-end code.

CONTENTS

- | | |
|-----------------------------|----------------------------|
| 1. Overview & current gap | 9. Reporting endpoint |
| 2. Architecture principles | 10. Front-end fetch layer |
| 3. Data model | 11. Waitlist |
| 4. Status lifecycle | 12. Packages & memberships |
| 5. Admin screens | 13. Settings endpoints |
| 6. Availability cross-check | 14. WhatsApp integration |
| 7. Computing open slots | 15. Build roadmap |
| 8. Demand reporting | 16. Seed data |
| | 17. Future considerations |

01

Overview & current gap

The site currently captures booking *requests* through three channels — an online form that posts to `/api/booking`, a pre-filled WhatsApp message to the clinic, and a phone number.

That is request capture, not a booking system. None of the three reserves a slot, checks availability, or holds a single source of truth; a human turns requests into appointments by hand. The most fragile point is the confirmation loop: the "real"

booking depends on the customer tapping the WhatsApp link *after* submitting the form, so many submissions become orphaned, and staff reconcile two channels manually.

This guide describes the system that closes those gaps: one record that advances through statuses, an availability engine that prevents double-booking, an admin dashboard staff run the day from, and a reporting view that shows where demand is high and low.

Core idea: a request and a confirmed booking are the **same database row** at different status values — not two tables. That single decision is what lets every screen edit the same object.

02

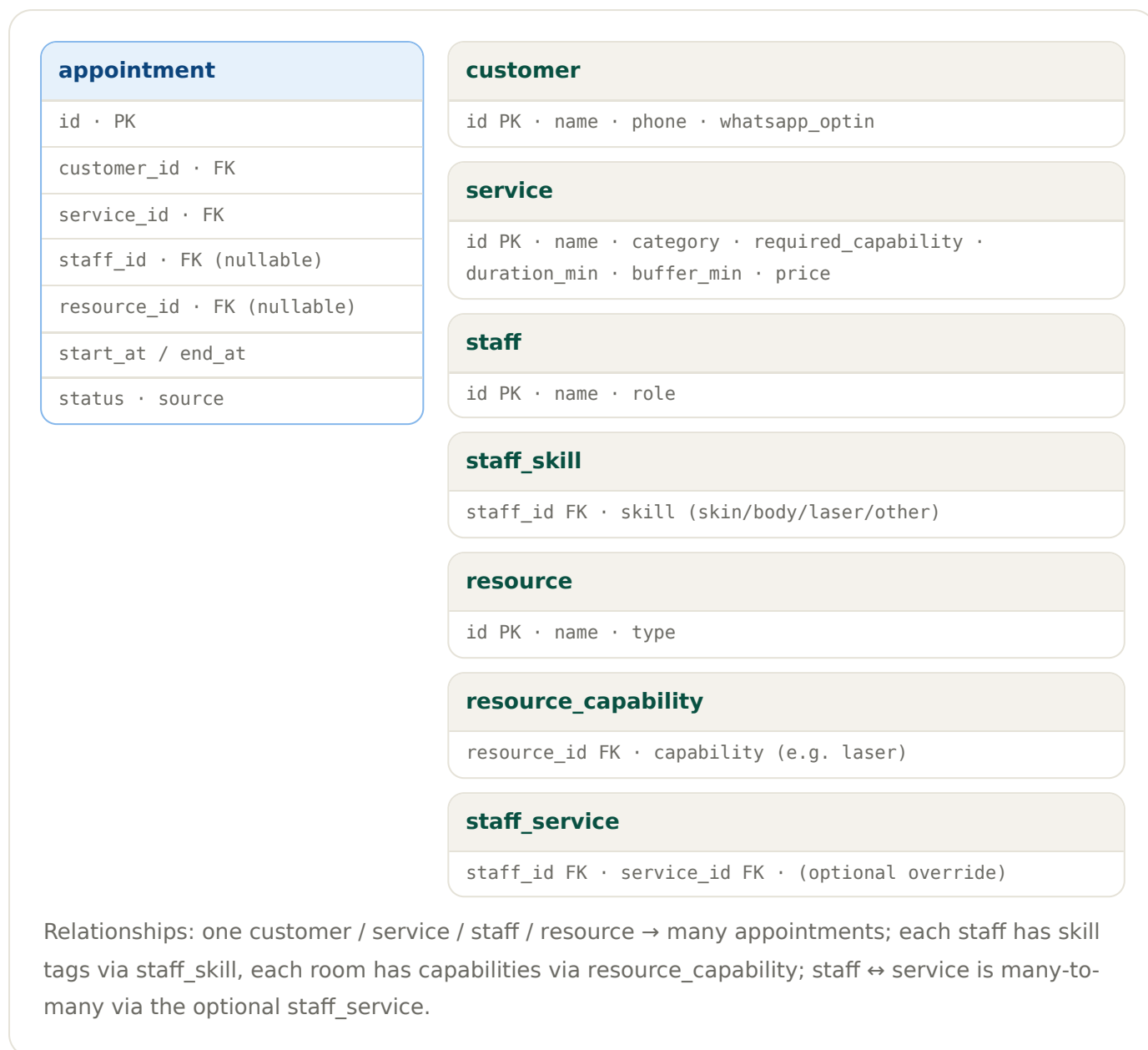
Architecture principles

- **Compute availability, don't store empty slots.** Derive open times from working hours minus existing appointments minus breaks, rather than pre-creating slot rows.
- **A booking needs a person *and* a resource.** Most appointments occupy both a practitioner and a room or machine at the same time. Model multi-resource from day one.
- **Buffers are part of the slot.** A 30-minute service with 10-minute cleanup occupies 40 minutes of the resource.
- **Guarantee no double-booking at the database, not just the app.** A read-time query gives good UX; a write-time constraint makes collisions impossible under a race.
- **Store everything in UTC;** convert to local only for display.
- **WhatsApp-first** is the right channel for this market — automate confirmations rather than relying on a manual deep-link.

03

Data model

The appointment table is the centre of gravity. A fresh request has a customer, a service and a tentative start_at, but staff_id and resource_id are null until confirmation. The staff_service join answers "who can perform this?", driving the practitioner dropdown.



Entity-relationship diagram. One central appointment linked to customer, service, staff and resource, plus the staff↔service skills join.

TABLE	PURPOSE & KEY COLUMNS
appointment	The booking. status, source (channel), start_at/end_at (end includes the buffer), nullable staff_id/resource_id.
customer	Name, phone, WhatsApp opt-in. Clinical notes live in a separate patient_record table, not here.
service	category (the skill it needs), required_capability (machine a room must have; null = any), duration_min, buffer_min, price.
staff / resource	Practitioners (with gender, for client preference matching) and rooms/machines.
staff_skill	Coarse skill tags per therapist (skin/body/laser/other) — the primary filter for who can be assigned. Edited on the Team & rooms screen.
staff_service	Optional per-service override for when skill tags are too coarse.
resource_capability	What a room/machine can do (e.g. laser) — narrows the room dropdown for services that require it.
staff_availability / staff_time_off	Working hours and breaks — drives the calendar grid (not shown in the diagram for clarity).


```

CREATE TABLE customer (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text NOT NULL,
  phone       text NOT NULL,
  whatsapp_optin boolean NOT NULL DEFAULT true,
  created_at  timestampz NOT NULL DEFAULT now()
);

CREATE TABLE service (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text NOT NULL,
  category    text NOT NULL,          -- 'skin'|'body'|'laser'|'other' – the
skill it needs
  required_capability text,          -- e.g. 'laser'; a room must have it. NULL
= any room
  duration_min int NOT NULL,
  buffer_min   int NOT NULL DEFAULT 0,
  price        numeric(10,2) NOT NULL,
  active       boolean NOT NULL DEFAULT true
);

CREATE TABLE staff (
  id      uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name    text NOT NULL,
  role    text,
  gender  text NOT NULL DEFAULT 'female', -- 'female' | 'male' – drives client
preference matching
  active  boolean NOT NULL DEFAULT true
);

CREATE TABLE resource (          -- rooms and machines
  id      uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name    text NOT NULL,
  type    text NOT NULL,          -- 'room' | 'machine'
  active  boolean NOT NULL DEFAULT true
);

CREATE TABLE resource_capability ( -- what a room/machine can do, e.g. 'laser'
  resource_id uuid REFERENCES resource(id),
  capability  text NOT NULL,
  PRIMARY KEY (resource_id, capability)
);

CREATE TABLE staff_skill (        -- coarse skill tags – the primary assignment
filter
  staff_id uuid REFERENCES staff(id),
  skill    text NOT NULL,          -- 'skin'|'body'|'laser'|'other'

```

```

PRIMARY KEY (staff_id, skill)
);

CREATE TABLE staff_service (  -- OPTIONAL: per-service override when skills are
too coarse
    staff_id    uuid REFERENCES staff(id),
    service_id  uuid REFERENCES service(id),
    PRIMARY KEY (staff_id, service_id)
);

CREATE TABLE staff_availability (  -- weekly working hours; drives the calendar
grid
    id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    staff_id    uuid REFERENCES staff(id),
    weekday     int  NOT NULL,          -- Postgres DOW: 0=Sun .. 6=Sat
    start_time  time NOT NULL,
    end_time    time NOT NULL
);

CREATE TABLE staff_time_off (      -- breaks, holidays, blocked time
    id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    staff_id    uuid REFERENCES staff(id),
    start_at    timestampz NOT NULL,
    end_at      timestampz NOT NULL,
    reason      text
);

CREATE TABLE appointment (
    id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id uuid NOT NULL REFERENCES customer(id),
    service_id  uuid NOT NULL REFERENCES service(id),
    staff_id    uuid REFERENCES staff(id),          -- null until confirmed
    resource_id uuid REFERENCES resource(id),       -- null until confirmed
    start_at    timestampz NOT NULL,
    end_at      timestampz NOT NULL,               -- start + duration + buffer
    status      text NOT NULL DEFAULT 'requested',
    source      text NOT NULL,                    --
'web_form'|'whatsapp'|'phone'|'walk_in'
    preferred_staff_gender text,                  -- 'female'|'male'|NULL = no
preference
    notes       text,
    created_at  timestampz NOT NULL DEFAULT now(),
    confirmed_at timestampz
);

-- Audit trail of every status change
CREATE TABLE appointment_event (
    id          bigserial PRIMARY KEY,

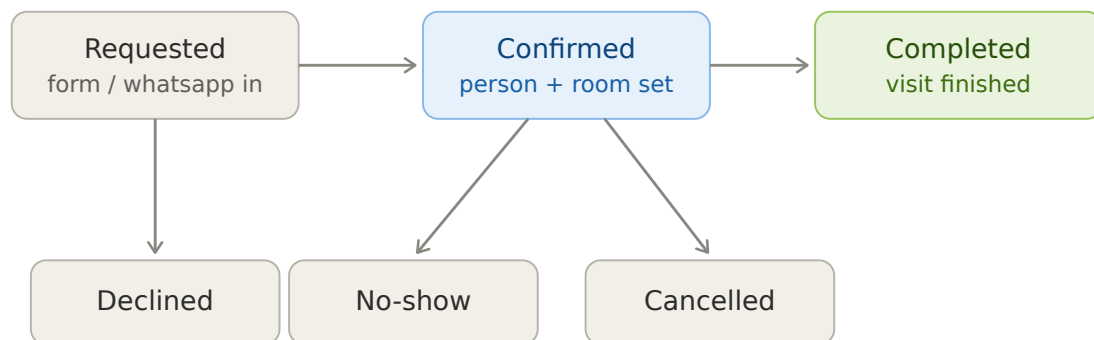
```

```
appointment_id uuid REFERENCES appointment(id),
from_status    text,
to_status      text,
actor          text,
created_at     timestampz NOT NULL DEFAULT now()
);
```

04

Status lifecycle

The whole system hinges on one transition: requested → confirmed. When a receptionist confirms, three things happen in one transaction — the `staff_id`, `resource_id` and final times are written; status flips to `confirmed` (activating the exclusion constraints that reserve the slot); and the WhatsApp confirmation fires. Until that flip, a request reserves nothing.



From requested a row can advance to confirmed or be declined; a confirmed booking ends as completed, no-show, or cancelled. Rescheduling is an edit to a confirmed row, not a new status.

Record every transition in `appointment_event` — it is cheap and lets you answer "who confirmed this and when," and later compute time-to-confirm and no-show rates.

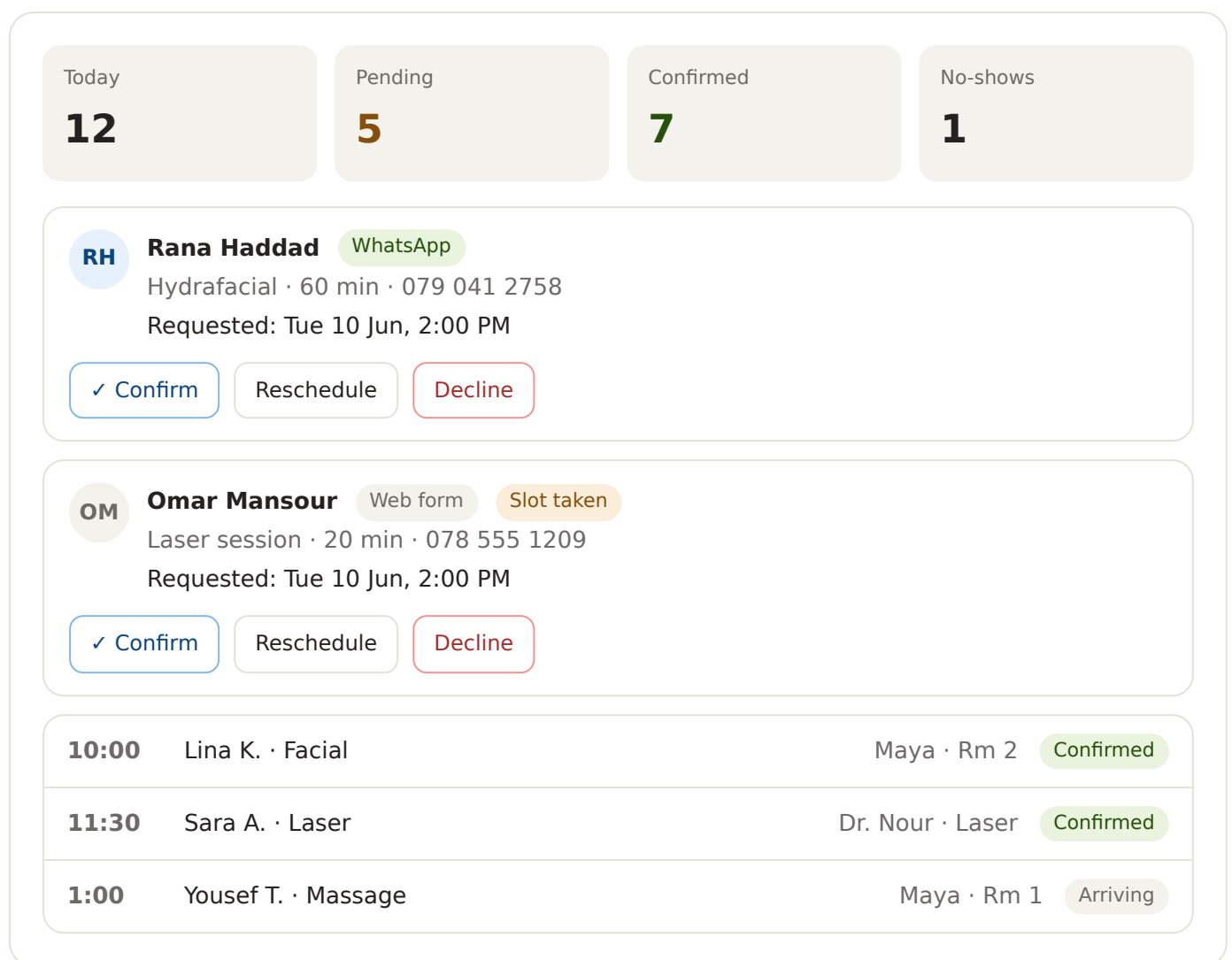
05

Admin screens

Three screens carry the daily work. The dashboard answers "what needs my attention now"; the request-detail view turns a loose request into a confirmed appointment; the calendar is the operational timeline.

Dashboard

Metric cards give the daily pulse; the pending-requests queue is the star, with channel badges and one-tap Confirm / Reschedule / Decline / WhatsApp actions.



Dashboard: summary metrics, the pending-requests queue, and today's schedule. The amber "Pending" count and the badge on requests pull attention to what's waiting.

Request detail

Opening a request lets staff assign service, practitioner, time and room. A live availability bar confirms the chosen person and room are both free; the primary button commits the booking *and* notifies the customer.

RH

Rana Haddad WhatsApp

079 041 2758 · Returning · 3 past visits

Pending

Customer requested
Hydrafacial · around 2:00 PM, Tue 10 Jun

Service

Hydrafacial — 60 min

Practitioner

Maya (esthetician)

Date

Tue 10 Jun

Time

2:00 PM

✓ Maya and Room 2 are both free at 2:00 PM

Confirm & notify on WhatsApp

Pending

Decline

Request detail: the green bar is the moment double-booking is prevented; confirming reserves the slot and sends the WhatsApp confirmation.

Calendar — day view

One column per practitioner, time down the side, each block sized to its real duration. Empty cells are clickable to create walk-ins; breaks render as first-class blocks.

	Maya	Dr. Nour	Lara
9:00			Color Dina M.
9:30			
10:00	Facial Lina K.		
10:30			
11:00			
11:30		Laser · Sara A.	Lunch
12:00	Massage Yousef T.	Hydrafacial Rana H.	
12:30			

Day view: block height equals duration; colour encodes status (green confirmed, amber pending, grey break). Confirming on the detail screen drops a block here.

Team & rooms (settings)

Register therapists with their skills and your rooms or machines. The skills ticked here are exactly what filter the practitioner dropdown when a request comes in — a laser booking only offers therapists carrying the Laser skill, and a skin service only offers Skin-skilled staff.

THERAPISTS & SKILLS

M

Maya

Skin

Body

L

Lara

Skin

Body

DN

Dr. Nour

Laser

New therapist name

Skin

Body

Laser

Other

+ Add

ROOMS & MACHINES

Treatment room 1

Room

Treatment room 2

Room

Laser room

laser

Machine

New room or machine name

Room ▼

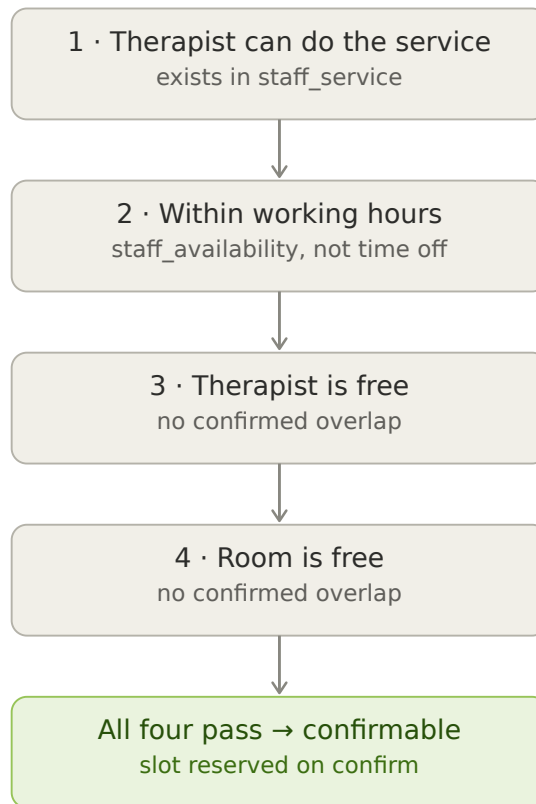
+ Add

Settings screen: therapists with their skill tags (skin / body / laser / other) and rooms / machines. Editing skills here immediately changes who the booking screen offers for each service category — no code change.

06

Availability cross-check

Before a booking can confirm, a proposed slot passes four gates, in order. Gates 3 and 4 share one overlap test, run once on the therapist and once on the room.



Two time windows clash when one starts before the other ends and vice versa. :new_end already includes the cleanup buffer.

Gate 1 is a skill match. Each service carries a category (skin / body / laser / other) and each therapist carries skill tags in `staff_skill` — the values set on the Team & rooms screen. The dropdown query intersects the two, so adding a skill to a therapist immediately widens who can be assigned, with no code change. Gate 1 also enforces **client gender preference**: in this market many clients will only accept a same-gender practitioner, so the booking carries an optional `preferred_staff_gender` and the filter honors it. A pairing the client would refuse at the door is not "available" — which is why this belongs in the availability engine, not in a later phase.

SQL · GATE 1 VIA SKILLS + GENDER PREFERENCE

```
-- Therapists whose skills cover the category the service needs,  
-- respecting the client's gender preference if one was given  
SELECT s.id, s.name  
FROM staff s  
JOIN staff_skill sk ON sk.staff_id = s.id  
JOIN service svc ON svc.id = :service_id  
WHERE sk.skill = svc.category -- e.g. a 'laser' service only offers Laser-  
skilled staff  
AND (:preferred_gender::text IS NULL OR s.gender = :preferred_gender)  
AND s.active;
```

The room dropdown narrows the same way through gate 4. If the service sets `required_capability` (a laser session needs the laser machine), only rooms whose `resource_capability` includes it appear; a service with no requirement matches any active room. The free-slot test is identical to the therapist overlap, run on `resource_id`.

SQL · ROOMS AVAILABLE FOR A SERVICE AT A TIME (GATE 4 + CAPABILITY)

```
SELECT r.id, r.name  
FROM resource r  
JOIN service svc ON svc.id = :service_id  
WHERE r.active  
AND (svc.required_capability IS NULL -- any room is fine, OR  
OR EXISTS ( -- the room has the  
capability  
SELECT 1 FROM resource_capability rc  
WHERE rc.resource_id = r.id  
AND rc.capability = svc.required_capability))  
AND NOT EXISTS ( -- gate 4: no room clash  
SELECT 1 FROM appointment a  
WHERE a.resource_id = r.id  
AND a.status = 'confirmed'  
AND a.start_at < :new_end  
AND a.end_at > :new_start);
```

SQL · THERAPIST OVERLAP CHECK (GATE 3; GATE 4 SWAPS STAFF_ID → RESOURCE_ID)

```
SELECT 1
FROM appointment
WHERE staff_id = :staff_id
      AND status = 'confirmed'
      AND id <> :current_appointment_id    -- skip self when rescheduling
      AND start_at < :new_end              -- existing starts before new ends
      AND end_at  > :new_start;            -- and ends after new starts
```

SQL · THERAPISTS AVAILABLE FOR A SERVICE AT A TIME (POPULATES THE DROPDOWN)

```
SELECT s.id, s.name
FROM staff s
JOIN staff_service ss
  ON ss.staff_id = s.id
  AND ss.service_id = :service_id        -- gate 1: qualified
JOIN staff_availability wa
  ON wa.staff_id = s.id
  AND wa.weekday = :weekday              -- gate 2: working that day
  AND wa.start_time <= :slot_start_time
  AND wa.end_time  >= :slot_end_time
WHERE s.active
      AND NOT EXISTS (                  -- gate 3: no therapist clash
        SELECT 1 FROM appointment a
        WHERE a.staff_id = s.id
              AND a.status = 'confirmed'
              AND a.start_at < :new_end
              AND a.end_at  > :new_start)
      AND NOT EXISTS (                  -- not on time off
        SELECT 1 FROM staff_time_off t
        WHERE t.staff_id = s.id
              AND t.start_at < :new_end
              AND t.end_at  > :new_start);
```

The query is the UX; the write-time guarantee is the exclusion constraint, which makes double-booking impossible even when two staff confirm the last slot at the same instant. You want both.

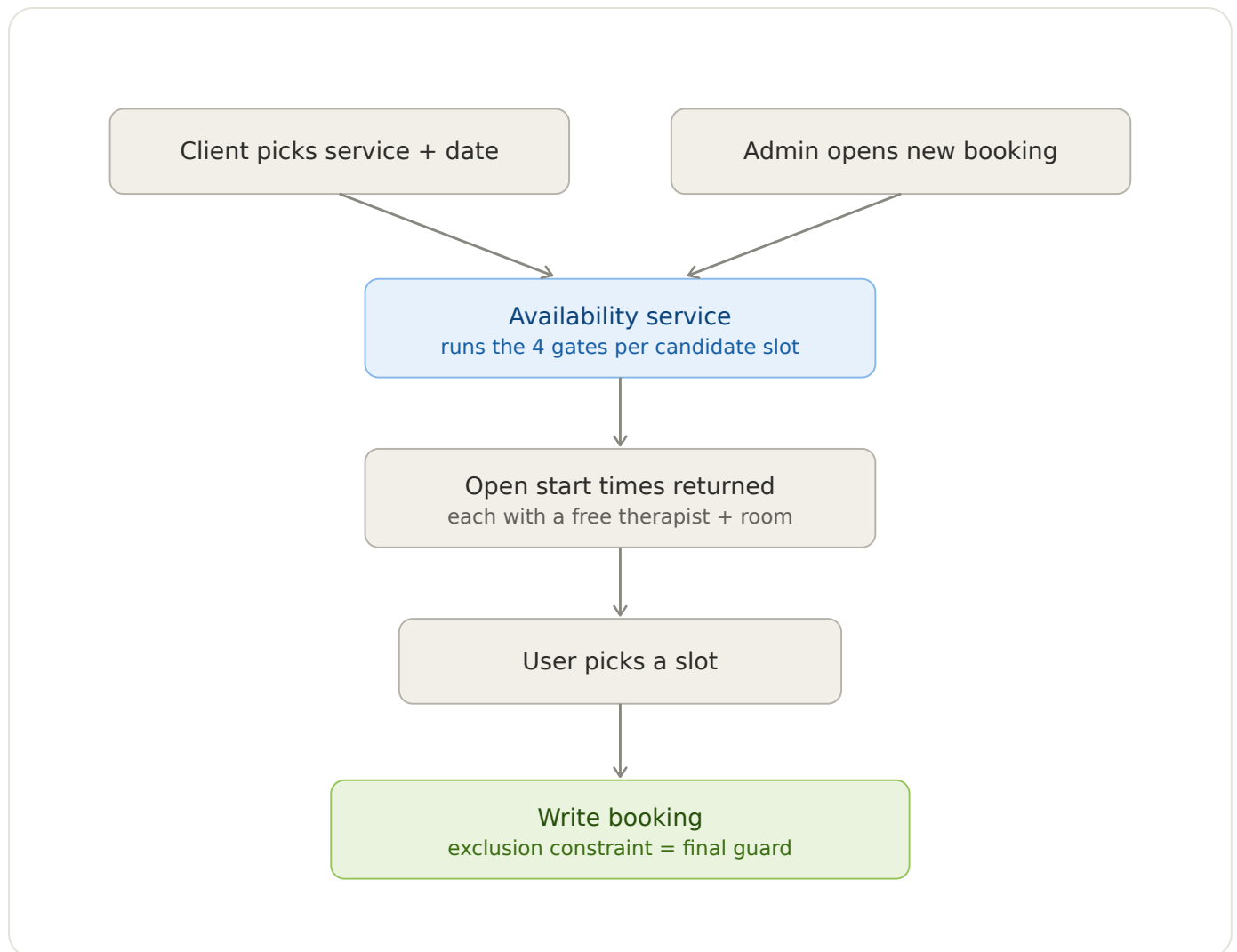
```
ALTER TABLE appointment ADD CONSTRAINT no_staff_overlap
EXCLUDE USING gist (
  staff_id WITH =,
  tstzrange(start_at, end_at) WITH &&
) WHERE (status = 'confirmed');

ALTER TABLE appointment ADD CONSTRAINT no_room_overlap
EXCLUDE USING gist (
  resource_id WITH =,
  tstzrange(start_at, end_at) WITH &&
) WHERE (status = 'confirmed');
```

07

Computing open slots

Availability is never stored — it is computed on demand. Whether a client books on the public form or an admin adds a booking, both call the same availability service, which runs the four gates from §6 against live data and returns only the start times that are actually free.



Both entry points call one availability service; it returns open start times, each already carrying a free therapist and room. The booking write is guarded by the exclusion constraints from §6.

For a service and a date, it derives the open times like this:

- Take the service's `duration_min + buffer_min` — the minutes each slot occupies (the buffer is included, so cleanup time is reserved).
- Find the qualified therapists (skill match) and the capable rooms (capability match).
- For each therapist, load their working hours for that weekday and their existing confirmed and offered appointments — the offered ones are waitlist holds and must block too.
- Walk the working window in small steps (e.g. 15 min). A candidate start is offered only if the full window fits inside a working block, the therapist has no overlap, and at least one capable room is free.

- Collapse to the distinct start times the customer can pick — each carrying a therapist and room that would serve it.


```

const SLOT_MIN = 15;

// GET /api/availability?service_id=&date=&therapist_id=(optional)
async function getAvailability(serviceId, date, onlyTherapistId = null) {
  const svc = (await db.query(
    `SELECT duration_min, buffer_min, category, required_capability
    FROM service WHERE id = $1`, [serviceId])).rows[0];
  const total = svc.duration_min + svc.buffer_min;    // minutes the slot
occupies

  // gate 1 - qualified therapists (optionally just the one the admin chose)
  const therapists = (await db.query(`
    SELECT s.id FROM staff s
    JOIN staff_skill sk ON sk.staff_id = s.id
    WHERE s.active AND sk.skill = $1 AND ($2::uuid IS NULL OR s.id = $2)`,
    [svc.category, onlyTherapistId])).rows;

  // capable rooms for this service (gate 4's capability half)
  const rooms = (await db.query(`
    SELECT r.id FROM resource r
    WHERE r.active AND ($1::text IS NULL
    OR EXISTS (SELECT 1 FROM resource_capability rc
    WHERE rc.resource_id = r.id AND rc.capability = $1))`,
    [svc.required_capability])).rows.map(r => r.id);

  const dow = new Date(date).getUTCDay();           // 0=Sun .. 6=Sat
  const slots = [];

  for (const t of therapists) {
    const hours = (await db.query(                  // gate 2 - working hours
      `SELECT start_time, end_time FROM staff_availability WHERE staff_id=$1 AND
weekday=$2`,
      [t.id, dow])).rows;
    const busy = (await db.query(                  // gate 3 - their day
      (holds included)
      `SELECT start_at, end_at FROM appointment
      WHERE staff_id=$1 AND status IN ('confirmed','offered') AND start_at::date
= $2`,
      [t.id, date])).rows;

    for (const block of hours) {
      let cursor = at(date, block.start_time);
      const blockEnd = at(date, block.end_time);
      while (addMin(cursor, total) <= blockEnd) {    // slot must fit the
working block
        const end = addMin(cursor, total);
        const therapistFree = !busy.some(b => b.start_at < end && b.end_at >

```

```

cursor);
    if (therapistFree) {
        const room = await firstFreeRoom(rooms, cursor, end);    // gate 4 - a
room is free
        if (room) slots.push({ start: cursor, therapist_id: t.id, resource_id:
room });
    }
    cursor = addMin(cursor, SLOT_MIN);
}
}
}
return dedupeByStart(slots);    // distinct pickable times; keep one
therapist+room per time
}

// first room with no overlap for the window
async function firstFreeRoom(roomIds, start, end) {
    const { rows } = await db.query(`
        SELECT r AS id FROM unnest($1::uuid[]) AS r
        WHERE NOT EXISTS (SELECT 1 FROM appointment a
            WHERE a.resource_id = r AND a.status IN ('confirmed','offered')
            AND a.start_at < $3 AND a.end_at > $2)
        LIMIT 1`, [roomIds, start, end]);
    return rows[0]?.id;
}
// at(date, time) and addMin(ts, mins) are small UTC date helpers.

```

The two entry points

Client booking: the public form calls `GET /api/availability?service_id=&date=` and shows only the returned times — no blind picking. Choosing one already comes with a therapist and room, so the booking auto-assigns them.

Admin adding a booking: the request-detail screen calls the same endpoint, optionally passing `therapist_id` to fill the time dropdown for one therapist. The admin sees only valid slots and can still override to force an assignment.

The final guard

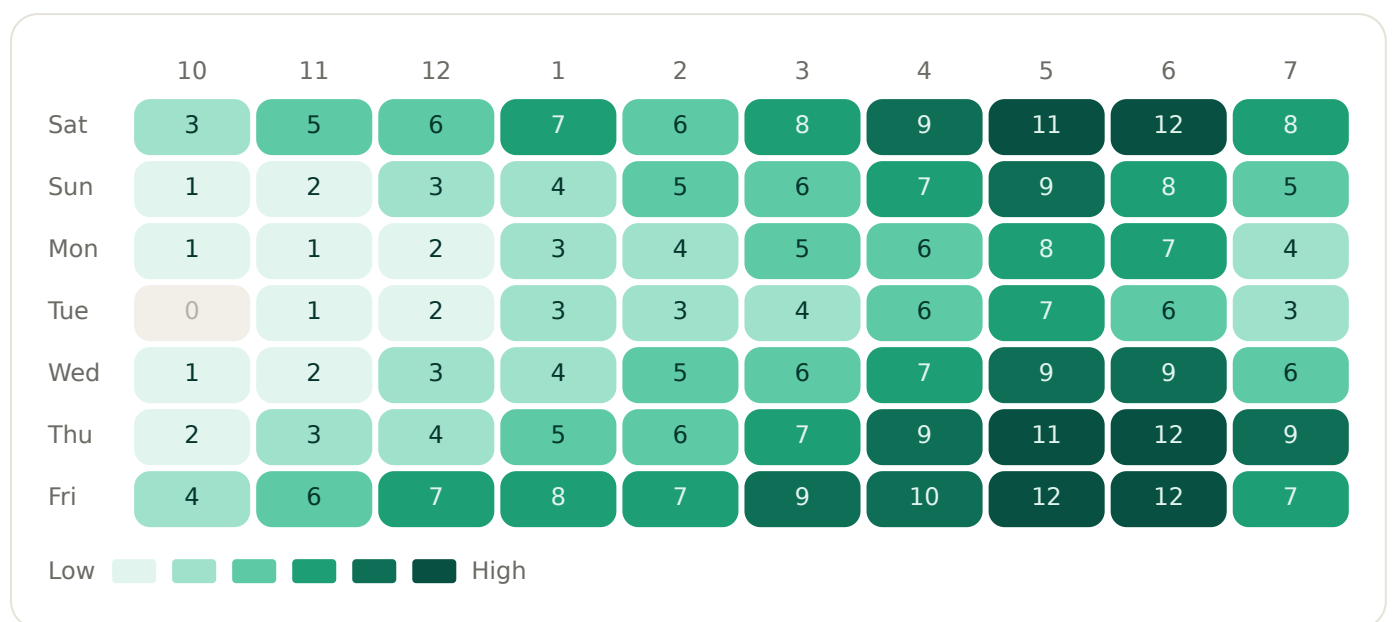
Availability is a snapshot — between showing a slot and submitting, someone else could take it. So the write re-runs the check inside a transaction, and the exclusion constraints from §6 are the real guarantee: if the slot was taken in those seconds, the

insert fails and the UI says the time was just booked. The service gives a clean experience; the constraint makes it correct under a race.

08

Demand reporting

The reporting view answers two questions: *when* are you busy vs quiet, and *which services* pull the most demand. Both come from the same appointment table — just two `GROUP BY`s over a date range.

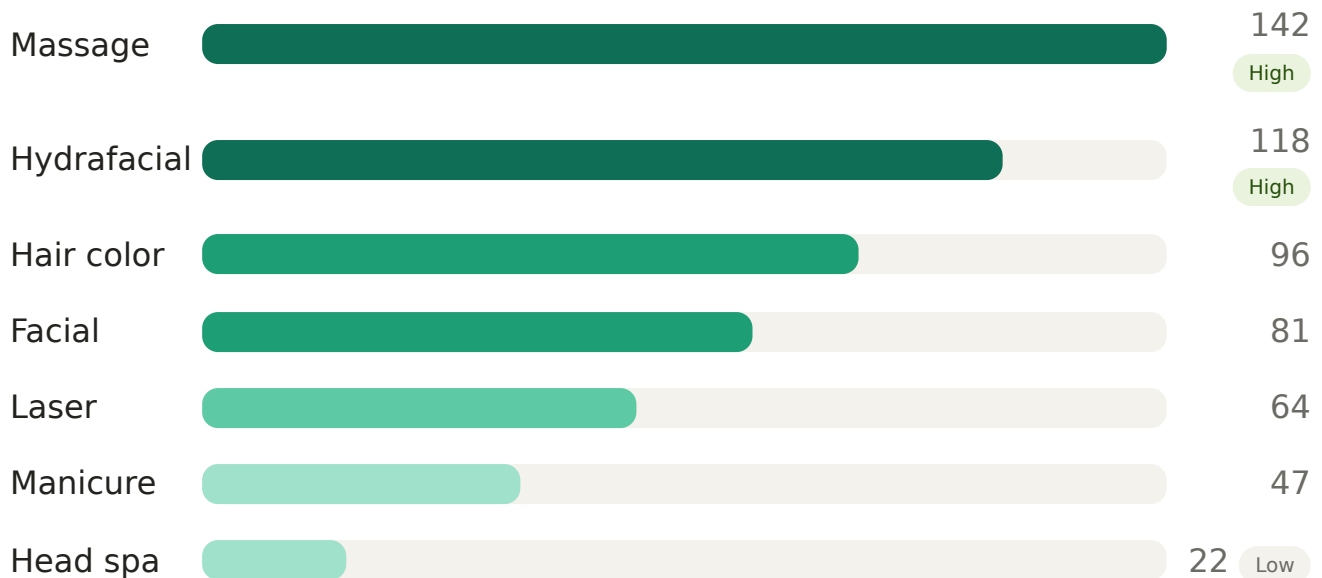


Demand heatmap by day-of-week $\times$ hour. Evenings (5–7pm) and the Fri/Sat weekend are the hot windows; weekday mornings are quiet — the case for off-peak promotions.

SQL · HEATMAP (GROUP BY WEEKDAY AND HOUR)

```
SELECT EXTRACT(DOW FROM start_at) AS weekday,
       EXTRACT(HOUR FROM start_at) AS hour,
       COUNT(*) AS requests
FROM appointment
WHERE start_at >= now() - interval '30 days'
GROUP BY weekday, hour
ORDER BY weekday, hour;
```

Requests, last 30 days



Demand by service, ranked high to low.

SQL · SERVICE RANKING

```
SELECT s.name, COUNT(*) AS requests
FROM appointment a
JOIN service s ON s.id = a.service_id
WHERE a.start_at >= now() - interval '30 days'
GROUP BY s.name
ORDER BY requests DESC;
```

Measure demand, not just fulfilment. Counting only status = 'confirmed' hides the slots you turned away. To see true demand, count every request regardless of final status; to see lost demand, filter to status IN ('declined', 'cancelled') — usually worst in the busy evenings, which is the business case for adding capacity.

09

Reporting endpoint

One endpoint serves every version of the grid. Grouping and status are whitelisted (never interpolated from raw input); filter values use parameterised placeholders, so there is no injection surface even though the query is assembled dynamically.

```

// GET /api/reports/demand
//   ?from=&to=&group_by=day_hour|service|channel|therapist
//   &status=requested|confirmed&therapist_id=&channel=

const GROUPINGS = {
  day_hour: "EXTRACT(DOW FROM a.start_at), EXTRACT(HOUR FROM a.start_at)",
  service: "s.name",
  channel: "a.source",
  therapist: "st.name",
};

app.get('/api/reports/demand', async (req, res) => {
  const { from, to } = req.query;
  const groupBy = GROUPINGS[req.query.group_by] ? req.query.group_by : 'day_hour';
  const status = req.query.status === 'confirmed' ? 'confirmed' : 'requested';

  const where = ['a.start_at >= $1', 'a.start_at < $2'];
  const params = [from, to];

  if (status === 'confirmed') where.push("a.status = 'confirmed'");
  if (req.query.therapist_id) { params.push(req.query.therapist_id);
  where.push(`a.staff_id = ${params.length}`); }
  if (req.query.channel) { params.push(req.query.channel);
  where.push(`a.source = ${params.length}`); }

  const sql = `
    SELECT ${GROUPINGS[groupBy]} AS bucket_key, COUNT(*) AS requests
    FROM appointment a
    JOIN service s ON s.id = a.service_id
    LEFT JOIN staff st ON st.id = a.staff_id -- LEFT: unassigned requests keep
counting
    WHERE ${where.join(' AND ')}
    GROUP BY ${GROUPINGS[groupBy]}
    ORDER BY ${GROUPINGS[groupBy]}`;

  const { rows } = await db.query(sql, params);
  res.json({ group_by: groupBy, status, from, to, buckets: rows });
});

```

For day_hour the JSON returns a flat list of { bucket_key: [dow, hour], requests }, which the front-end pivots into the 7×10 grid. For the other groupings bucket_key is a single label feeding the ranked-bar view. One endpoint, four reports.

Front-end fetch layer

A framework-free module that fetches the report live. It handles four states — loading skeleton, empty, error with retry, and success — and uses an `AbortController` to cancel stale requests when filters change quickly, so a slow response can't overwrite a newer one.

HTML · MOUNT

```
<div id="report"></div>
<script type="module">
  import { initDemandReport } from './demand-report.js';
  initDemandReport(document.getElementById('report'), {
    endpoint: '/api/reports/demand',
    therapists: [
      { id: 'st_1', name: 'Maya' },
      { id: 'st_2', name: 'Lara' },
      { id: 'st_3', name: 'Dr. Nour' },
    ],
  });
</script>
```



```

const DAYS = ['Sat', 'Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri'];
const HOURS = [10, 11, 12, 13, 14, 15, 16, 17, 18, 19];

// Postgres EXTRACT(DOW): 0=Sun .. 6=Sat. Map to our row order (Sat first).
const DOW_TO_ROW = { 6: 0, 0: 1, 1: 2, 2: 3, 3: 4, 4: 5, 5: 6 };
const SCALE = ['#E1F5EE', '#9FE1CB', '#5DCAA5', '#1D9E75', '#0F6E56', '#085041'];
const CHANNELS = [
  { value: 'all', label: 'All channels' },
  { value: 'whatsapp', label: 'WhatsApp' },
  { value: 'form', label: 'Web form' },
  { value: 'phone', label: 'Phone' },
];

function isoDaysAgo(days) {
  const d = new Date();
  d.setDate(d.getDate() - days);
  return d.toISOString().slice(0, 10);
}

function fillColor(n, max) {
  if (n === 0) return '#f1efe8';
  const r = n / max;
  if (r <= 0.17) return SCALE[0];
  if (r <= 0.34) return SCALE[1];
  if (r <= 0.50) return SCALE[2];
  if (r <= 0.67) return SCALE[3];
  if (r <= 0.84) return SCALE[4];
  return SCALE[5];
}

function textColor(n, max) {
  if (n === 0) return '#b4b2a9';
  return n / max > 0.5 ? '#E1F5EE' : '#04342C';
}

function pivot(buckets) {
  const grid = DAYS.map(() => HOURS.map(() => 0));
  for (const b of buckets) {
    const [dow, hour] = b.bucket_key;
    const row = DOW_TO_ROW[dow];
    const col = HOURS.indexOf(hour);
    if (row != null && col >= 0) grid[row][col] = Number(b.requests) || 0;
  }
  return grid;
}

export function initDemandReport(container, opts = {}) {
  const endpoint = opts.endpoint || '/api/reports/demand';
  const therapists = opts.therapists || [];
  const state = { status: 'requested', rangeDays: 30, channel: 'all', therapistId:

```

```

'all' };
let inflight = null;

container.innerHTML = `... controls, summary, body, legend (see demand-report.js
file) ...`;
const summaryEl = container.querySelector('.dr-summary');
const bodyEl = container.querySelector('.dr-body');

function renderSkeleton() { summaryEl.textContent = 'Loading...'; /* grey pulsing
grid */ }
function renderError(retry) { /* message + Try-again button calling retry */ }
function renderEmpty() { /* "No bookings in this range" */ }

function renderGrid(buckets) {
  const grid = pivot(buckets);
  let total = 0, max = 0, peak = null;
  for (let d = 0; d < 7; d++) for (let h = 0; h < 10; h++) {
    const v = grid[d][h]; total += v;
    if (v > max) { max = v; peak = { d, h }; }
  }
  if (total === 0) return renderEmpty();
  if (max === 0) max = 1;
  // build summary + 7x10 grid with fillColor/textColor relative to max
}

function buildUrl() {
  const p = new URLSearchParams({
    from: isoDaysAgo(state.rangeDays), to: isoDaysAgo(0),
    group_by: 'day_hour', status: state.status,
  });
  if (state.channel !== 'all') p.set('channel', state.channel);
  if (state.therapistId !== 'all') p.set('therapist_id', state.therapistId);
  return `${endpoint}?${p.toString()}`;
}

async function load() {
  if (inflight) inflight.abort(); // cancel stale request
  inflight = new AbortController();
  const signal = inflight.signal;
  renderSkeleton();
  try {
    const res = await fetch(buildUrl(), { signal, headers: { Accept:
'application/json' } });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const data = await res.json();
    if (signal.aborted) return;
    renderGrid(data.buckets || []);
  } catch (err) {

```

```
    if (err.name === 'AbortError') return; // expected on filter change
    renderError(load);
  }
}
// wire status toggle, range, channel, therapist selects -> load()
load();
}
```

Integration notes. The therapist dropdown sends `therapist_id` (pass real staff IDs; names are display-only). The pivot assumes `bucket_key = [dow, hour]` with Postgres DOW semantics (0 = Sunday), reordered by `D0W_T0_R0W` to put Saturday first — adjust if your week starts elsewhere. `HOURS` is hard-coded 10:00–19:00. For auth behind a session cookie add `credentials: 'include'` to the fetch options. The complete, runnable module ships alongside this guide as `demand-report.js`.

11

Waitlist

When a customer wants a slot that's already full — or any slot on a busy evening — they join a waitlist for a service and a preferred window instead of being turned away. When a confirmed booking is cancelled or marked no-show, the freed slot is automatically offered to matching waiting customers over WhatsApp, first to accept wins. This is how you fill the dark cells in the demand heatmap.

A 2:00 PM Fri slot just opened — 3 people are waiting for Hydrafacial

Offer to next

WAITING LIST

RH

Rana Haddad

Hydrafacial · prefers Fri afternoon · waiting 2 days

Offer slot

SK

Sami Khoury

Hydrafacial · any evening · waiting 1 day

Offer slot

NA

Nadia A.

Hydrafacial · prefers Maya · waiting 5 hours

Offered

Waitlist view: when a slot frees, the admin (or an automated job) offers it to waiting customers in order. "Offered" entries hold the slot briefly until the customer accepts or it times out.

The waitlist extends the data model from §3 with one table:

SQL · WAITLIST TABLE

```
CREATE TABLE waitlist (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id uuid NOT NULL REFERENCES customer(id),
  service_id  uuid NOT NULL REFERENCES service(id),
  staff_id    uuid REFERENCES staff(id),    -- optional preferred therapist
  preferred_from timestamptz,              -- earliest acceptable time (NULL =
anytime)
  preferred_to  timestamptz,              -- latest acceptable time
  status        text NOT NULL DEFAULT 'waiting', --
waiting|offered|converted|expired|cancelled
  held_appointment_id uuid REFERENCES appointment(id), -- the slot held while an
offer is out
  offered_at    timestamptz,              -- when the current offer was
sent
  created_at    timestamptz NOT NULL DEFAULT now()
);
```

When an appointment is cancelled, the freed window (its start_at/end_at, staff and room) is matched against waiting entries for the same service, oldest first:

```
SELECT w.id, w.customer_id
FROM waitlist w
WHERE w.service_id = :service_id
      AND w.status = 'waiting'
      AND (w.staff_id IS NULL OR w.staff_id = :freed_staff_id)
      AND (w.preferred_from IS NULL OR w.preferred_from <= :freed_start)
      AND (w.preferred_to IS NULL OR w.preferred_to >= :freed_end)
ORDER BY w.created_at      -- first come, first served
LIMIT 1;
```

On offer, set the entry to offered and send a time-limited WhatsApp accept link. If accepted, create the appointment (status confirmed) and mark the entry converted; if it times out, advance to the next person. Waitlist depth is also a clean demand signal — counting waiting entries by day and hour mirrors the heatmap and shows exactly where you're turning business away.

Auto-offer worker

A small background worker turns the waitlist into something automatic. It reacts to a cancellation, holds the freed slot as a tentative offered appointment (so no one else can grab it while the offer is out), messages the next waiter, and rolls on to the next person if the offer expires. The hold reuses the appointment table — extend the exclusion constraints' filter to status IN ('confirmed', 'offered') so held slots reserve the therapist and room too, and deleting the hold releases them.

Four pieces: an entry point on cancellation, the offer step, the customer's accept, and a periodic sweeper for expired offers.


```

const OFFER_TTL_MIN = 20;

// 1) Entry point – fired when an appointment is cancelled or marked no-show.
// Its slot (service, staff, room, time) is now open.
async function onAppointmentFreed(slot) {      // {service_id, staff_id,
resource_id, start_at, end_at}
  await offerNextWaiter(slot);
}

// 2) Offer the freed slot to the next eligible waiter (FIFO).
async function offerNextWaiter(slot) {
  const { rows } = await db.query(`
    SELECT id, customer_id FROM waitlist
    WHERE service_id = $1 AND status = 'waiting'
      AND (staff_id IS NULL OR staff_id = $2)
      AND (preferred_from IS NULL OR preferred_from <= $3)
      AND (preferred_to IS NULL OR preferred_to >= $4)
    ORDER BY created_at LIMIT 1`,
    [slot.service_id, slot.staff_id, slot.start_at, slot.end_at]);
  if (!rows.length) return;                // nobody waiting – leave the
slot open
  const entry = rows[0];

  // Hold the slot as a tentative appointment so it can't be double-booked while
the offer is out.
  const held = await db.query(`
    INSERT INTO appointment (customer_id, service_id, staff_id, resource_id,
start_at, end_at, status, source)
    VALUES ($1,$2,$3,$4,$5,$6,'offered','waitlist') RETURNING id`,
    [entry.customer_id, slot.service_id, slot.staff_id, slot.resource_id,
slot.start_at, slot.end_at]);

  await db.query(`UPDATE waitlist SET status='offered', offered_at=now(),
held_appointment_id=$2 WHERE id=$1`,
    [entry.id, held.rows[0].id]);

  await whatsapp.sendTemplate(entry.customer_id, 'waitlist_offer', {
    when: formatLocal(slot.start_at),
    link: acceptUrl(signOffer(entry.id)),      // single-use link, valid
OFFER_TTL_MIN minutes
  });
}

// 3) Customer taps the accept link in time.
async function acceptOffer(entryId) {
  await db.tx(async t => {
    const { rows } = await t.query(

```

```

    `SELECT held_appointment_id FROM waitlist WHERE id=$1 AND status='offered'`,
    [entryId]);
    if (!rows.length) throw new Error('offer expired');
    await t.query(`UPDATE appointment SET status='confirmed', confirmed_at=now()
WHERE id=$1`,
    [rows[0].held_appointment_id]);
    await t.query(`UPDATE waitlist SET status='converted' WHERE id=$1`,
    [entryId]);
  });
  // then send the normal booking confirmation
}

// 4) Sweeper (cron, every few minutes) – expire stale offers and roll to the next
person.
async function expireStaleOffers() {
  const { rows } = await db.query(`
    SELECT w.id, a.service_id, a.staff_id, a.resource_id, a.start_at, a.end_at,
w.held_appointment_id
    FROM waitlist w JOIN appointment a ON a.id = w.held_appointment_id
    WHERE w.status='offered' AND w.offered_at < now() - ($1 * interval '1
minute')`,
    [OFFER_TTL_MIN]);
  for (const r of rows) {
    await db.query(`DELETE FROM appointment WHERE id=$1`,
    [r.held_appointment_id]); // release the hold
    await db.query(`UPDATE waitlist SET status='expired' WHERE id=$1`, [r.id]);
    // this waiter missed it
    await offerNextWaiter(r);
    // offer to the next
  }
}

```

The accept step runs inside a transaction and flips the existing held appointment to confirmed, so the slot was reserved the whole time — there is no window where two waiters could both accept. If a real booking arrives for the same slot while an offer is out, the exclusion constraint blocks it because the hold already occupies the slot.

12

Packages & memberships

Clinics run on bundles — a six-session laser package, or a monthly facial membership. You sell a balance, track sessions remaining against the customer, and

redeem one each time a booking for that service completes. This changes the revenue model, not just the booking flow.

CUSTOMER PACKAGE BALANCE

RH

Rana Haddad · Laser package

package

3 of 6 sessions left · expires 12 Aug 2026

ON THE BOOKING SCREEN

Redeem from Laser package (3 of 6 left)

Applied

CATALOGUE

Laser — 6 sessions

package

450 JOD

Monthly facial

membership

60 JOD / mo

Packages: a customer's balance with sessions remaining, the redeem option surfaced on the booking screen, and the sellable catalogue. Memberships are unlimited within their period rather than session-counted.

Three additions to the data model from §3 — a catalogue, a per-customer balance, and a link from the appointment that consumed a session:

```

CREATE TABLE package (
    id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    name        text NOT NULL,
    service_id  uuid REFERENCES service(id), -- scope to one service, OR
    category    text,                      -- scope to a category
    (skin/body/laser)
    total_sessions int,                    -- NULL = unlimited (membership)
    price       numeric(10,2) NOT NULL,
    valid_days  int,                      -- expiry window from purchase
    kind        text NOT NULL DEFAULT 'package' -- 'package' | 'membership'
);

CREATE TABLE customer_package (
    id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id uuid NOT NULL REFERENCES customer(id),
    package_id  uuid NOT NULL REFERENCES package(id),
    sessions_left int,                    -- NULL = unlimited membership
    purchased_at timestampz NOT NULL DEFAULT now(),
    expires_at  timestampz,
    status      text NOT NULL DEFAULT 'active' -- active|used_up|expired
);

-- link the appointment that consumed a session
ALTER TABLE appointment
    ADD COLUMN customer_package_id uuid REFERENCES customer_package(id);

```

On the booking screen, offer any active package that covers the service:

```

SELECT cp.id, p.name, cp.sessions_left
FROM customer_package cp
JOIN package p ON p.id = cp.package_id
WHERE cp.customer_id = :customer_id
    AND cp.status = 'active'
    AND (cp.expires_at IS NULL OR cp.expires_at > now())
    AND (cp.sessions_left IS NULL OR cp.sessions_left > 0)
    AND (p.service_id = :service_id OR p.category = :service_category);

```

Decrement the balance when the visit is marked completed (run inside the same status transition). Memberships have a NULL balance and are never decremented; a cancelled-before-completion booking simply leaves the balance untouched:

```
UPDATE customer_package
SET sessions_left = sessions_left - 1,
    status = CASE WHEN sessions_left - 1 <= 0 THEN 'used_up' ELSE status END
WHERE id = :customer_package_id
AND sessions_left IS NOT NULL;      -- memberships (NULL) are unlimited
```

13

Settings endpoints

The Team & rooms screen needs write endpoints to persist. `POST /api/staff` creates a therapist and their skills in one transaction; `POST /api/rooms` does the same for a room and its capabilities. Both validate input against the allowed values, and deletes are soft (set `active = false`) so historical appointments still resolve.


```

const SKILLS      = ['skin', 'body', 'laser', 'other'];
const CAPABILITIES = ['laser']; // extend as you add machines

// POST /api/staff  { name, role, skills: ['skin','laser'] }
app.post('/api/staff', async (req, res) => {
  const { name, role, skills = [] } = req.body;
  if (!name || !name.trim()) return res.status(400).json({ error: 'name required'
});
  const bad = skills.filter(s => !SKILLS.includes(s));
  if (bad.length) return res.status(400).json({ error: 'unknown skills: ' +
bad.join(', ') });

  const staff = await db.tx(async t => {
    const { rows } = await t.query(
      `INSERT INTO staff (name, role) VALUES ($1,$2) RETURNING id, name, role`,
      [name.trim(), role || null]);
    for (const skill of skills) {
      await t.query(`INSERT INTO staff_skill (staff_id, skill) VALUES ($1,$2)
        ON CONFLICT DO NOTHING`, [rows[0].id, skill]);
    }
    return { ...rows[0], skills };
  });
  res.status(201).json(staff);
});

// POST /api/rooms  { name, type: 'room'|'machine', capabilities: ['laser'] }
app.post('/api/rooms', async (req, res) => {
  const { name, type = 'room', capabilities = [] } = req.body;
  if (!name || !name.trim()) return res.status(400).json({ error: 'name required'
});
  if (!['room', 'machine'].includes(type)) return res.status(400).json({ error:
'invalid type' });
  const bad = capabilities.filter(c => !CAPABILITIES.includes(c));
  if (bad.length) return res.status(400).json({ error: 'unknown capabilities: ' +
bad.join(', ') });

  const room = await db.tx(async t => {
    const { rows } = await t.query(
      `INSERT INTO resource (name, type) VALUES ($1,$2) RETURNING id, name, type`,
      [name.trim(), type]);
    for (const cap of capabilities) {
      await t.query(`INSERT INTO resource_capability (resource_id, capability)
VALUES ($1,$2)
        ON CONFLICT DO NOTHING`, [rows[0].id, cap]);
    }
    return { ...rows[0], capabilities };
  });
});

```

```

    res.status(201).json(room);
  });

  // Soft delete – keep the row so past appointments still resolve
  app.delete('/api/staff/:id', async (req, res) => {
    await db.query(`UPDATE staff SET active = false WHERE id = $1`,
      [req.params.id]);
    res.status(204).end();
  });

```

The settings screen wires straight to these: the skill chips become the `skills` array sent to `POST /api/staff`, and the list query's aggregated `skills` render the pills back.

SQL • LIST ACTIVE STAFF WITH THEIR SKILLS

```

SELECT s.id, s.name, s.role,
       COALESCE(array_agg(sk.skill) FILTER (WHERE sk.skill IS NOT NULL), '{}') AS
skills
FROM staff s
LEFT JOIN staff_skill sk ON sk.staff_id = s.id
WHERE s.active
GROUP BY s.id, s.name, s.role
ORDER BY s.name;

```

14

WhatsApp integration

The system leans on WhatsApp throughout — confirmations, reminders, and waitlist offers. Replace the manual deep-link with the WhatsApp Business Cloud API: pre-approved outbound templates plus an inbound webhook for replies and button taps (the waitlist accept). One thin helper sends every message; a small wrapper names the templates the rest of the code calls.

```

// Sends a pre-approved template message via the WhatsApp Cloud API
async function sendTemplate(toPhone, template, params, lang = 'ar') {
  const r = await
  fetch(`https://graph.facebook.com/v20.0/${PHONE_NUMBER_ID}/messages`, {
    method: 'POST',
    headers: { Authorization: `Bearer ${WHATSAPP_TOKEN}`, 'Content-Type':
'application/json' },
    body: JSON.stringify({
      messaging_product: 'whatsapp',
      to: toPhone, // E.164, e.g. 962790412758
      type: 'template',
      template: {
        name: template, // pre-approved in Meta Business
        language: { code: lang }, // 'ar' or 'en'
        components: [{ type: 'body', parameters: params.map(text => ({ type:
'text', text })) }],
      },
    })),
  });
  if (!r.ok) throw new Error('whatsapp send failed: ' + r.status);
  return r.json();
}

// Everything in the app calls these – never the raw API
const notify = {
  confirmed: (a) => sendTemplate(a.phone, 'booking_confirmation',
[a.customerName, a.serviceName, a.whenLocal]),
  reminder: (a, h) => sendTemplate(a.phone, h === 24 ? 'reminder_24h' :
'reminder_2h', [a.customerName, a.whenLocal]),
  offer: (e, link) => sendTemplate(e.phone, 'waitlist_offer', [e.whenLocal,
link]),
  declined: (a) => sendTemplate(a.phone, 'request_declined',
[a.customerName]),
};

```

The inbound webhook handles Meta's one-time verification and incoming replies — including the button tap that accepts a waitlist offer, which calls the `acceptOffer` worker from §11.

```
// GET – Meta webhook verification (once, at setup)
app.get('/webhooks/whatsapp', (req, res) => {
  if (req.query['hub.verify_token'] === VERIFY_TOKEN) return
  res.send(req.query['hub.challenge']);
  res.sendStatus(403);
});

// POST – inbound messages and button replies
app.post('/webhooks/whatsapp', async (req, res) => {
  res.sendStatus(200); // acknowledge fast, then process
  const msg = req.body?.entry?.[0]?.changes?.[0]?.value?.messages?.[0];
  if (!msg) return;
  if (msg.button?.payload?.startsWith('ACCEPT_')) { // waitlist "Accept"
    button
    await acceptOffer(msg.button.payload.slice('ACCEPT_'.length));
  }
});
```

Reminders are a scheduled job — a cron runs hourly, finds confirmed appointments coming up, and sends each one once. A tiny notification table dedupes sends so a retried job can't message twice.

```
CREATE TABLE notification ( -- one row per message sent; dedupes reminders
  appointment_id uuid REFERENCES appointment(id),
  kind text NOT NULL, -- 'reminder_24h' | 'reminder_2h' | ...
  sent_at timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (appointment_id, kind)
);

SELECT a.id, c.phone, c.name AS customer_name, a.start_at
FROM appointment a
JOIN customer c ON c.id = a.customer_id
WHERE a.status = 'confirmed'
  AND a.start_at BETWEEN now() + interval '23 hours' AND now() + interval '24
hours'
  AND NOT EXISTS (SELECT 1 FROM notification n
                  WHERE n.appointment_id = a.id AND n.kind = 'reminder_24h');
```

A few setup notes: every template (booking_confirmation, reminder_24h, waitlist_offer, ...) must be pre-approved in Meta Business; outside a pre-approved template you

can only send free text within the 24-hour customer-service window after the customer's last message. Store phone numbers in E.164 — for Jordan that means +962 with the leading zero dropped, so 079 041 2758 is sent as 962790412758.

15

Build roadmap

Sequenced so each stage is usable before the next begins.

STAGE	WHAT YOU BUILD
1 · Source of truth	Make /api/booking persist every submission as an appointment with a status and reference. Auto-confirm receipt to the customer. Add spam protection (rate limit + honeypot, ideally phone OTP).
2 · Admin core	Dashboard + pending-requests queue + request-detail view. Confirming writes staff/room/time, flips status, and fires the WhatsApp confirmation.
3 · Availability engine	The four-gate check and the two exclusion constraints. Practitioner/room dropdowns show only valid options.
4 · Calendar	Day view by practitioner; click empty cells for walk-ins; breaks as blocks. Mobile falls back to a scrollable day list with a staff filter.
5 · Reminders & no-shows	Automated 24h / 2h WhatsApp reminders; track no-shows per customer; deposits for high-value services.
6 · Reporting	The demand endpoint and live report screen with filters (requested vs confirmed, channel, therapist, date range).
7 · Waitlist & packages	Auto-offer freed slots to waiting customers over WhatsApp; sell and redeem session bundles and memberships against bookings.

Two decisions shape the rest: whether to keep building custom vs. adopt an off-the-shelf platform (Fresha, Boulevard, SimplyBook.me) that already does availability, reminders and deposits; and how strict the clinic-side data rules are — keep clinical notes in a separate, role-restricted table from the first commit.

16

Seed data

A concrete starting dataset so whoever builds it can run the queries immediately. It populates the catalogue, rooms and their capabilities, the team and their skills,

working hours, and two packages — consistent with the examples used throughout this guide.


```

-- Services (category = the skill needed; required_capability = the machine a room
must have)
INSERT INTO service (name, category, required_capability, duration_min,
buffer_min, price) VALUES
    ('Hydrafacial',    'skin',    NULL,    60, 10, 45.00),
    ('Classic facial', 'skin',    NULL,    45, 10, 30.00),
    ('Deep massage',  'body',    NULL,    60, 10, 40.00),
    ('Laser session', 'laser',    'laser', 20, 10, 35.00),
    ('Hair color',    'other',    NULL,    90, 15, 55.00),
    ('Manicure',      'other',    NULL,    45,  5, 20.00),
    ('Head spa',      'body',    NULL,    50, 10, 38.00);

-- Rooms & machines
INSERT INTO resource (name, type) VALUES
    ('Treatment room 1', 'room'),
    ('Treatment room 2', 'room'),
    ('Laser room',       'machine'),
    ('Hair station',     'room');

-- Capabilities: only the laser machine carries the 'laser' capability
INSERT INTO resource_capability (resource_id, capability)
SELECT id, 'laser' FROM resource WHERE name = 'Laser room';

-- Team (gender drives client preference matching)
INSERT INTO staff (name, role, gender) VALUES
    ('Maya',    'esthetician',    'female'),
    ('Lara',    'stylist',        'female'),
    ('Dr. Nour', 'laser technician', 'female'),
    ('Khaled',  'barber',         'male');

-- Skills (skin / body / laser / other)
INSERT INTO staff_skill (staff_id, skill)
SELECT st.id, v.skill
FROM staff st
JOIN (VALUES
    ('Maya','skin'), ('Maya','body'),
    ('Lara','skin'), ('Lara','other'),
    ('Khaled','other'),
    ('Dr. Nour','laser')
) AS v(name, skill) ON v.name = st.name;

-- Working hours: Sat–Thu, 10:00–19:00 for everyone (Postgres DOW:
6=Sat,0=Sun,1..4=Mon–Thu)
INSERT INTO staff_availability (staff_id, weekday, start_time, end_time)
SELECT st.id, d.weekday, time '10:00', time '19:00'
FROM staff st CROSS JOIN (VALUES (6),(0),(1),(2),(3),(4)) AS d(weekday);

```

```
-- Packages: a 6-session laser bundle and a monthly facial membership
INSERT INTO package (name, service_id, category, total_sessions, price,
valid_days, kind) VALUES
  ('Laser - 6 sessions', (SELECT id FROM service WHERE name='Laser session'),
'laser', 6, 180.00, 180, 'package'),
  ('Monthly facial', NULL,
'skin', NULL, 60.00, 30, 'membership');
```

With this loaded, a "Laser session" request offers only Dr. Nour (the Laser skill) in the laser room, a "Hydrafacial" offers Maya in either treatment room, and the laser bundle is redeemable for the laser service — exercising every rule in the guide end to end.

17

Future considerations

Patterns borrowed from platforms whose architecture mirrors this guide's design — Qmatic's enterprise appointment model and MioSalon's salon operations playbook. None is needed for launch; each is a small, additive extension when the business calls for it.

Resource pools

Treat interchangeable rooms as a pool and ask "is anything in the pool free?" rather than checking rooms one by one. The availability engine already behaves this way — `firstFreeRoom()` picks the first free room among the capable ones — so this is mostly a naming and admin-UI concept: rooms sharing the same capabilities are one pool ("Treatment rooms", "Laser"), and the admin assigns the pool, not a specific room, unless they want to override. If explicit pools become useful (e.g. pricing or reporting per pool), add:

```
CREATE TABLE resource_pool (
  id    uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name  text NOT NULL          -- 'Treatment rooms', 'Laser'
);
ALTER TABLE resource ADD COLUMN pool_id uuid REFERENCES resource_pool(id);
-- Availability then filters rooms by pool instead of by capability list.
```

Group bookings

Some services can take more than one customer at once (a slimming class, a group session). Today the overlap check assumes capacity 1. To support groups, give the service a capacity and replace the binary overlap test with a count:

```
ALTER TABLE service ADD COLUMN max_customers int NOT NULL DEFAULT 1;

-- A slot is open if confirmed bookings for this service/staff/time < capacity
SELECT COUNT(*) < svc.max_customers AS has_space
FROM service svc
LEFT JOIN appointment a
  ON a.service_id = svc.id
  AND a.staff_id  = :staff_id
  AND a.status    = 'confirmed'
  AND a.start_at  = :slot_start      -- group sessions share the exact
slot
WHERE svc.id = :service_id
GROUP BY svc.max_customers;
```

Note the exclusion constraints from §6 must then exempt group services (e.g. constrain only `max_customers = 1` services, or move the guarantee to a per-slot counter table), since multiple confirmed rows will legitimately share one staff/time range.

Multi-service appointments

A client books a facial *and* a manicure back-to-back in one flow, with one confirmation and one WhatsApp message. The schema already supports it — it is simply two consecutive appointment rows — so the work is UX plus a grouping key:

```
ALTER TABLE appointment ADD COLUMN booking_group_id uuid; -- same id across the chain
```

```
-- Availability for a chain: find a start time where each leg, placed
-- end-to-end (including buffers), passes the four gates for its own
-- service — possibly with different therapists per leg, same visit.
```

Confirm, reschedule, cancel, and notify then operate on the `booking_group_id` as a unit. This is the highest-value of the three for a spa — chained bookings raise average ticket value — and a natural Phase 3+ feature once single-service flow is smooth.

Google-review routing (feedback flow)

Borrowed from MioSalon's feedback design, and directly tied to local-SEO goals: after a completed visit, the WhatsApp flow asks for a 1–5 rating. A 5 gets the Google review link; a low rating alerts the manager instead — happy clients grow the public rating, unhappy ones are handled privately before they post. Reviews are a top local-pack ranking factor, so this small addition compounds.

```
CREATE TABLE feedback (
  appointment_id uuid PRIMARY KEY REFERENCES appointment(id),
  rating          int CHECK (rating BETWEEN 1 AND 5),
  comment         text,
  routed_to       text,          -- 'google_review' | 'manager_alert'
  created_at      timestampz NOT NULL DEFAULT now()
);
-- Webhook on rating reply:
--   rating = 5      -> send Google review link template;
routed_to='google_review'
--   rating <= 3     -> WhatsApp/notify manager with comment;
routed_to='manager_alert'
```

Graduated no-show policy

Deposits for everyone were deliberately skipped; this targets only proven offenders. At booking time, count the customer's past `no_show` rows — above a threshold, the

request requires upfront payment (or at minimum an explicit double-confirmation link) before it can be confirmed. 95% of clients book friction-free; the costly 5% are covered.

SQL • NO-SHOW GATE AT BOOKING TIME

```
SELECT COUNT(*) >= 2 AS require_prepayment      -- threshold: 2 prior no-shows
FROM appointment
WHERE customer_id = :customer_id AND status = 'no_show';
```

Service-interval rebooking reminders

Laser and many treatments are cyclical. Give each service a rebook interval; a daily job finds customers whose last completed session is due and sends a "time for your next session" WhatsApp. This drives package completion and repeat revenue with zero staff effort.

SQL • WHO IS DUE A REBOOKING NUDGE (DAILY CRON)

```
ALTER TABLE service ADD COLUMN rebook_interval_days int;    -- e.g. laser = 35

SELECT DISTINCT ON (a.customer_id, a.service_id)
    a.customer_id, a.service_id, a.start_at AS last_visit
FROM appointment a
JOIN service s ON s.id = a.service_id
WHERE a.status = 'completed'
    AND s.rebook_interval_days IS NOT NULL
    AND a.start_at < now() - (s.rebook_interval_days * interval '1 day')
    AND NOT EXISTS (
        -- no upcoming booking
        SELECT 1 FROM appointment n
        WHERE n.customer_id = a.customer_id AND n.service_id = a.service_id
            AND n.status IN ('requested', 'confirmed') AND n.start_at > now())
ORDER BY a.customer_id, a.service_id, a.start_at DESC;
-- Pair with a notification-log row (kind='rebook_nudge') so each cycle nudges
once.
```

Arrival check-in

The one useful piece of queue thinking for a bookings-first clinic: when a client walks in, reception marks the confirmed appointment as *arrived*. The dashboard's "Today's

schedule" then shows live status at a glance — who's here, who's in treatment, who's late — and walk-ins join the same view as bookings. It also makes the no-show decision objective: anything unarrived N minutes past its start time is flagged for follow-up or marked `no_show`.

SQL · ARRIVAL TIMESTAMP + LATE LIST

```
ALTER TABLE appointment ADD COLUMN arrived_at timestamptz;

-- Reception taps "Arrived" on the schedule row:
UPDATE appointment SET arrived_at = now() WHERE id = :id AND status = 'confirmed';

-- Late list for the dashboard (15+ min past start, not arrived):
SELECT a.id, c.name, a.start_at
FROM appointment a JOIN customer c ON c.id = a.customer_id
WHERE a.status = 'confirmed'
      AND a.arrived_at IS NULL
      AND a.start_at < now() - interval '15 minutes'
      AND a.start_at::date = current_date;
```

No new status is needed — `arrived_at` is orthogonal to the lifecycle, and a row that ends the day confirmed-but-unarrived is the candidate set for `no_show`.

Build guide for the Artizone Spa & Clinic booking system · all timestamps stored in UTC · data and figures are illustrative.